

UNIVERSIDADE FEDERAL DE ALAGOAS

Instituto de Computacao

Framework Whitelabel para Sistemas Online de Aluguel de Casas e Apartamentos

Relatorio Tecnico de Desenvolvimento

<https://github.com/TheMarksan/aluguel-services-framework>

Alunos:

Marcos Melo dos Santos

(Gateway, Reservas Core e Front-end via Figma/MCP)

Leonardo Barbosa

(Persistencia e logica interna de Auth e Catalogo)

Disciplina:

Reuso de Software e Metodologias Ageis

Periodo: 2026.1

Maceio – Alagoas
2026

Sumário

1	Enunciado do Problema	3
2	Planejamento Scrum por Prioridades	3
2.1	Metodologia Agil Escolhida: Scrum	3
2.2	Product Backlog	3
2.3	Sprint Backlog por Sprint	3
2.4	Justificativa das Prioridades	3
3	Tecnicas de Reuso de Software Aplicadas	5
4	Arquitetura da Solucao	5
4.1	Diagramas de Casos de Uso	6
4.2	Interfaces dos Componentes do Framework	9
4.3	Listagem dos Microservicos em Execucao	9
5	API Gateway	10
5.1	Especificacao Documentada do API Gateway	10
5.2	Microservico de Autenticacao (ms-auth)	11
5.3	Microservico de Catalogo (ms-catalogo)	11
5.4	Microservico de Reservas (ms-reservas)	12
6	Front-End Integrado	12
7	Tecnologias Utilizadas	13
7.1	Tecnologias dos Microservicos	13
7.2	Tecnologias do Framework e das Aplicacoes	13
8	Template Method e Hotspots	13
8.1	Classes Concretas e Prova de Reuso	14
8.2	Aplicacao 1: Aluguel por Temporada	15
8.3	Aplicacao 2: Aluguel de Longa Duracao	16
8.4	Como Criar uma Nova Aplicacao Reusando o Framework	16
9	Guia do Desenvolvedor	17
9.1	Pre-requisitos	17
9.2	Subindo os Servicos	17
9.3	Criando os Bancos	17
9.4	Como Estender o Framework	18

10 Funcionamento do Sistema	18
10.1 Passo a Passo das Telas	18
10.2 Exemplos de Execucao das Duas Aplicacoes	18
11 Prints do Sistema em Execucao	19
12 Repositorio GitHub	26
12.1 Observacao sobre Video	26
A Anexos:Codigo dos Microservicos	26
A.1 API Gateway	26
A.2 Microservico ms-auth	27
A.3 Microservico ms-catalogo	27
A.4 Microservico ms-reservas e Aplicacoes	28
Referencias	29

1 Enunciado do Problema

O presente trabalho tem como objetivo o desenvolvimento de um **Framework Whitelabel** para a construcao de sistemas online de aluguel de casas e apartamentos. Um produto *whitelabel* e aquele que pode ser reaproveitado e customizado por diferentes clientes (imobiliarias, plataformas de temporada, redes de coliving), mantendo um nucleo comum de regras de negocio.

O desafio central, do ponto de vista de **Reuso de Software**, e projetar uma arquitetura que maximize o reaproveitamento do fluxo de reserva, ao mesmo tempo em que permita variacoes especificas de cada modalidade de locacao. Para isso, adota-se o padrao de projeto *Template Method*, que define o esqueleto invariante do algoritmo de reserva e delega passos variaveis (*hotspots*) para subclasses.

O sistema e estruturado como um **monorepo** de microservicos em PHP 8.2 e MySQL, contemplando autenticacao, catalogo de imoveis e a engine de reservas, integrados por um *API Gateway*.

2 Planejamento Scrum por Prioridades

2.1 Metodologia Agil Escolhida: Scrum

A metodologia agil escolhida foi o **Scrum**, adaptado para uma dupla e para o calendario da disciplina. O projeto foi dividido em sprints curtas, cada uma com uma meta objetiva, um incremento entregavel e registro em branch/commit. O acompanhamento foi feito por backlog priorizado, validacao incremental e revisao do incremento ao final de cada etapa.

No contexto do trabalho, os papeis foram simplificados: a dupla atuou como time de desenvolvimento; os requisitos da disciplina funcionaram como *Product Owner*; e a revisao de cada entrega funcionou como validacao da sprint. Essa adaptacao manteve o foco principal do Scrum: entregar valor em ciclos curtos, inspecionar o que foi feito e ajustar a proxima etapa.

2.2 Product Backlog

O desenvolvimento seguiu uma abordagem incremental, organizada em um *Product Backlog* priorizado. Cada item foi tratado como uma *Sprint* curta, com entrega versionada ao final de cada etapa.

2.3 Sprint Backlog por Sprint

2.4 Justificativa das Prioridades

Os itens de infraestrutura e seguranca (Gateway e Autenticacao) recebem prioridade alta por serem dependencias transversais. A engine de reservas, sendo o nucleo de reuso e o diferencial academico do trabalho, tambem possui prioridade alta. O catalogo e as classes concretas dependem dessas bases, e o front-end e a camada de apresentacao final, com prioridade mais baixa por nao bloquear as demais entregas.

Prioridade	Item do Backlog (User Story)	Status
1 (Alta)	Inicializacao do repositorio: <code>.gitignore</code> , <code>README</code> e relatorio tecnico.	Concluido
2 (Alta)	API Gateway: roteamento central <i>stateless</i> e comunicacao interna via <code>cURL</code> .	Concluido
3 (Alta)	Microservico de Autenticacao (<code>ms-auth</code>) com JWT e banco <code>db_auth</code> .	Concluido
4 (Media)	Microservico de Catalogo (<code>ms-catalogo</code>) e banco <code>db_catalogo</code> .	Concluido
5 (Alta)	Engine de Reservas (<code>ms-reservas</code>): classe abstrata <code>RentEngine</code> (Template Method).	Concluido
6 (Media)	Classes concretas <code>LongTermRent</code> e <code>VacationRent</code> , provando o reuso.	Concluido
7 (Baixa)	Front-end integrado no Gateway (Blade/HTML) consumindo o ecossistema.	Concluido

Tabela 1: Product Backlog priorizado do framework.

Sprint	Meta	Itens do Sprint Backlog	Entrega
1	Inicializar o monorepo	Criar estrutura de pastas; adicionar <code>README</code> ; preparar relatorio em LaTeX; definir escopo e arquitetura inicial.	Base do projeto
2	Implementar Gateway	Criar <code>gateway</code> ; configurar roteamento <code>/api/*</code> ; implementar cliente HTTP via <code>cURL</code> ; servir a view principal.	Gateway funcional
3	Implementar autenticacao	Criar <code>ms-auth</code> ; modelar usuarios; implementar cadastro, login, JWT, validacao de token e seed administrativo.	Auth JWT
4	Implementar catalogo	Criar <code>ms-catalogo</code> ; modelar imoveis; implementar CRUD, filtros, validacoes e vinculo logico com proprietario.	Catalogo REST
5	Implementar framework de reservas	Criar <code>ms-reservas</code> ; implementar <code>RentEngine</code> ; definir frozen spots e hotspots; criar persistencia de reservas.	Nucleo reutilizavel
6	Criar aplicacoes concretas	Implementar <code>VacationRent</code> ; implementar <code>LongTermRent</code> ; adicionar <code>CreditCheckComponent</code> ; registrar modalidades na fabrica.	Duas aplicacoes por reuso
7	Integrar interface e seguranca	Criar front-end integrado no Gateway; consumir Auth, Catalogo e Reservas; proteger rotas de escrita via validacao JWT.	Sistema integrado

Tabela 2: Sprint Backlog resumido por sprint.

3 Técnicas de Reuso de Software Aplicadas

O projeto aplica multiplas tecnicas de reuso, em diferentes niveis de granularidade:

Frameworks de Aplicacao O proprio produto e um *framework whitelabel*: fornece um esqueleto reutilizavel que e instanciado por diferentes aplicacoes de aluguel.

Padroes de Projeto Uso do *Template Method* para fixar o fluxo de reserva e variar passos especificos via *hotspots*.

Reuso por Heranca As modalidades `VacationRent` e `LongTermRent` herdam de `RentEngine`, reaproveitando o algoritmo comum.

Reuso por Composicao A funcionalidade `CreditCheckComponent` e injetada em `LongTermRent` por composicao, evitando heranca rigida e favorecendo flexibilidade.

Reuso de Servicos (SOA) A arquitetura de microservicos permite que autenticacao e catalogo sejam reutilizados por qualquer cliente do ecossistema.

Componentes de Software O `CreditCheckComponent` representa um componente plugavel do framework: ele encapsula uma regra especifica, e e usado por composicao somente pela aplicacao que precisa de analise de credito.

Assim, as tecnicas de reuso utilizadas foram: **framework de aplicacao**, **microservicos**, **componentes de software**, **Template Method**, **heranca** e **composicao**. O framework fornece o fluxo comum; os microservicos separam capacidades reutilizaveis; e os hotspots permitem adaptar o comportamento sem reescrever a engine.

4 Arquitetura da Solucao

A solucao adota uma arquitetura de **microservicos** em *monorepo*, com isolamento de dados (*Database per Service*). O cliente comunica-se exclusivamente com o Gateway, que orquestra as chamadas internas.

Servico	Porta	Banco	Responsabilidade
gateway	8003	—	Roteamento e composicao (cURL)
ms-auth	8000	db_auth	Autenticacao JWT
ms-catalogo	8001	db_catalogo	Catalogo de imoveis
ms-reservas	8002	db_reservas	Engine de reservas

Tabela 3: Mapeamento de servicos, portas e bancos de dados.

```
[ Cliente / Browser ]
  |
  v
[ API Gateway :8003 ] --- stateless, roteia via cURL
  | | |
```

```

v v v
ms-auth ms-catalogo ms-reservas
:8000 :8001 :8002
db_auth db_catalogo db_reservas

```

Listing 1: Topologia logica de comunicacao.

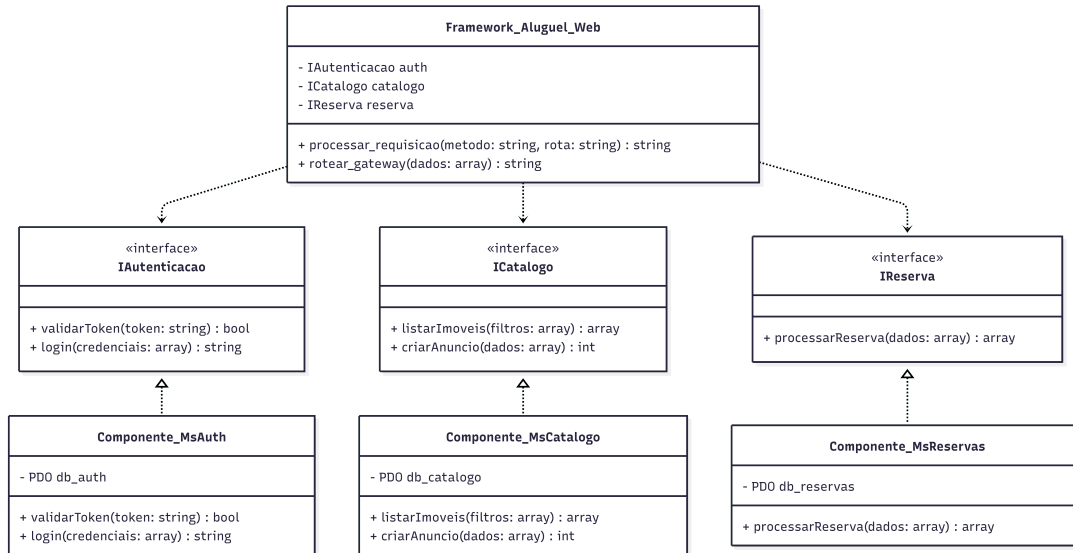


Figura 1: Diagrama de classes estrutural do framework, com a classe de framework e os componentes de software.

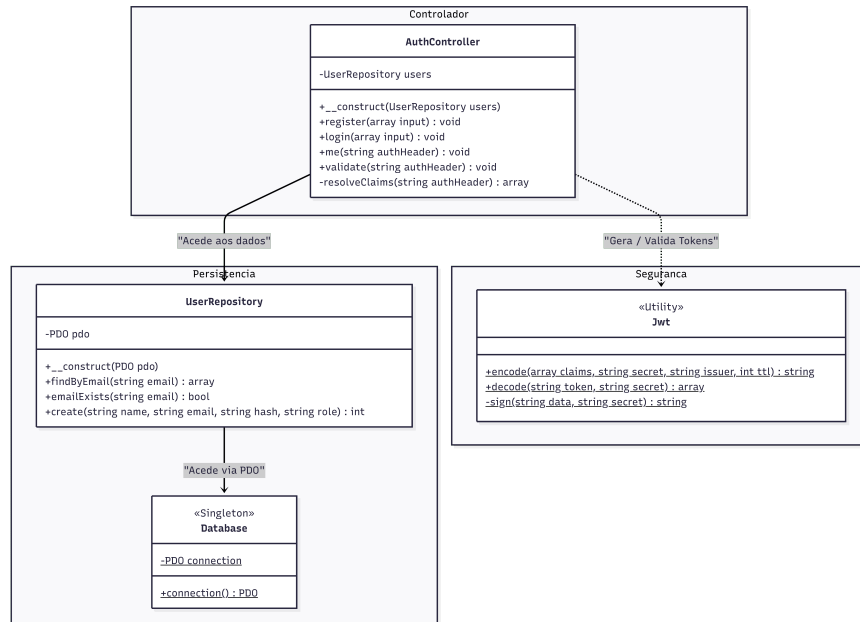


Figura 2: Diagrama de classes do componente de autenticação ms-auth.

4.1 Diagramas de Casos de Uso

Os diagramas de casos de uso mostram a visão funcional do sistema e do núcleo reutilizável do framework. No sistema completo, os atores acessam apenas o Gateway; a partir dele,

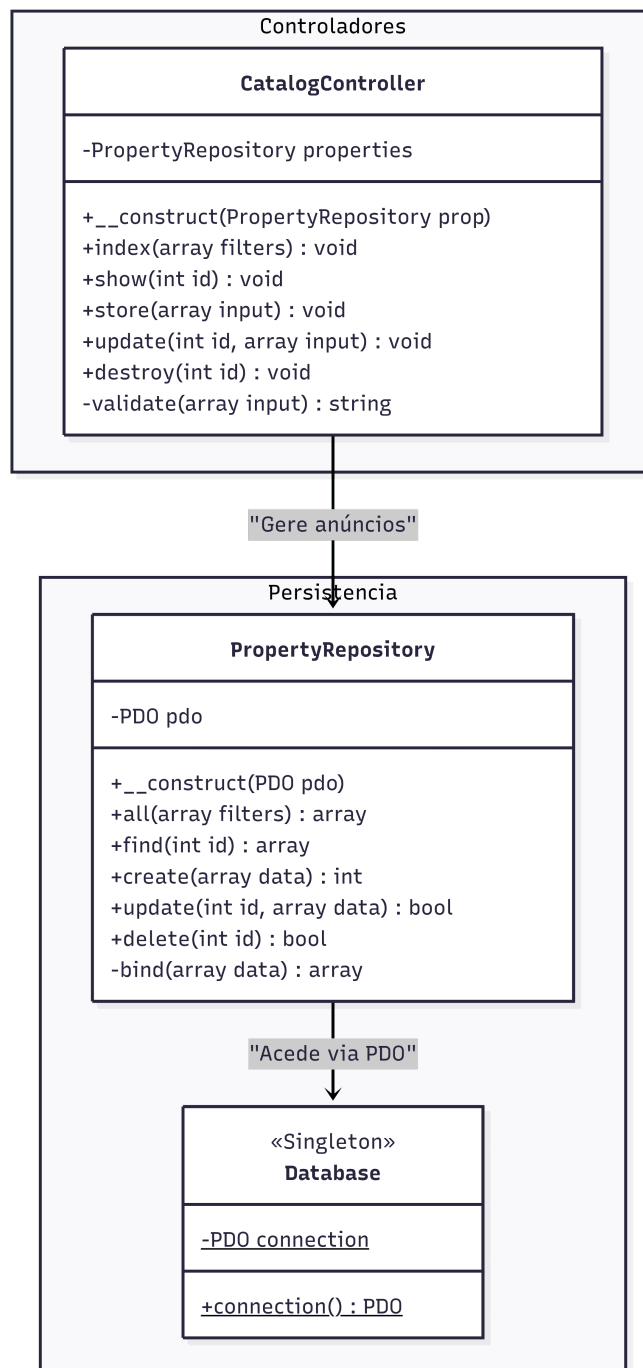


Figura 3: Diagrama de classes do componente de catalogo ms-catalogo.

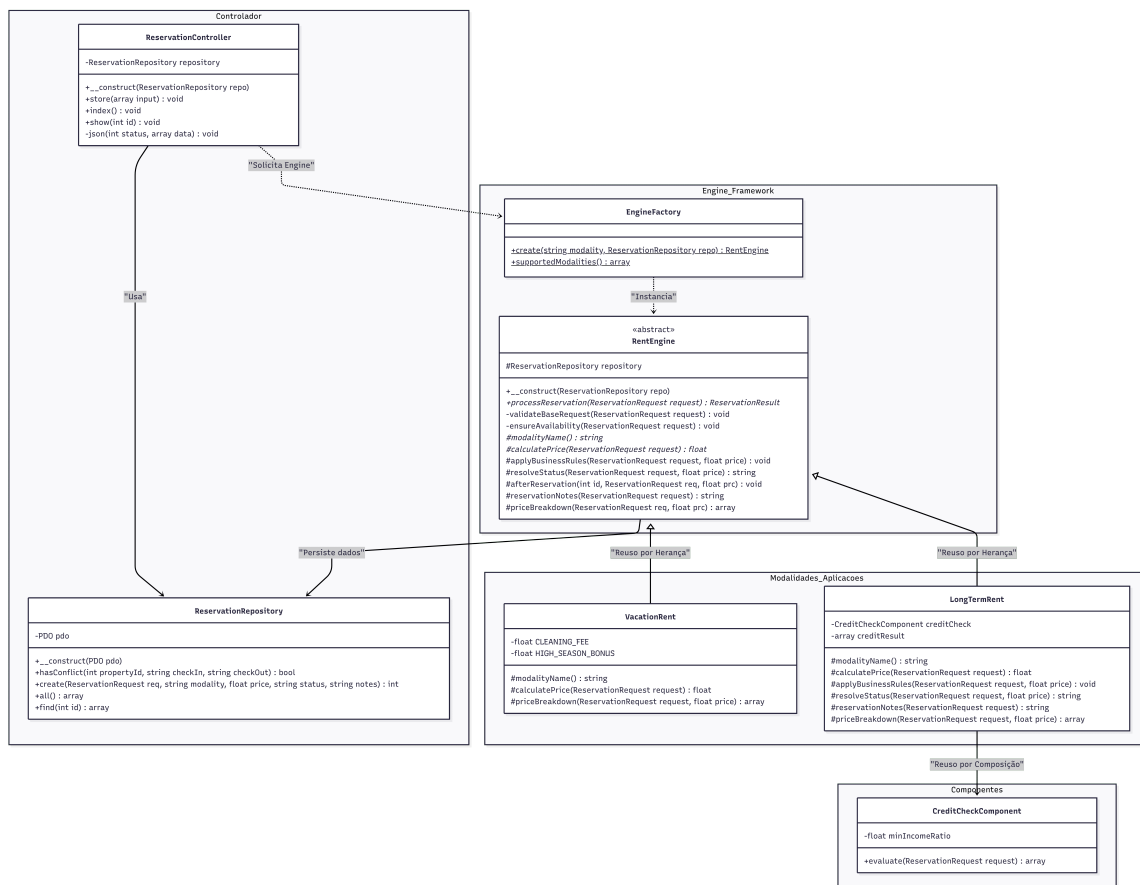


Figura 4: Diagrama de classes do componente de reservas, com Template Method, hotspots e modalidades concretas.

as funcionalidades acionam os microservicos de autenticao, catalogo e reservas.

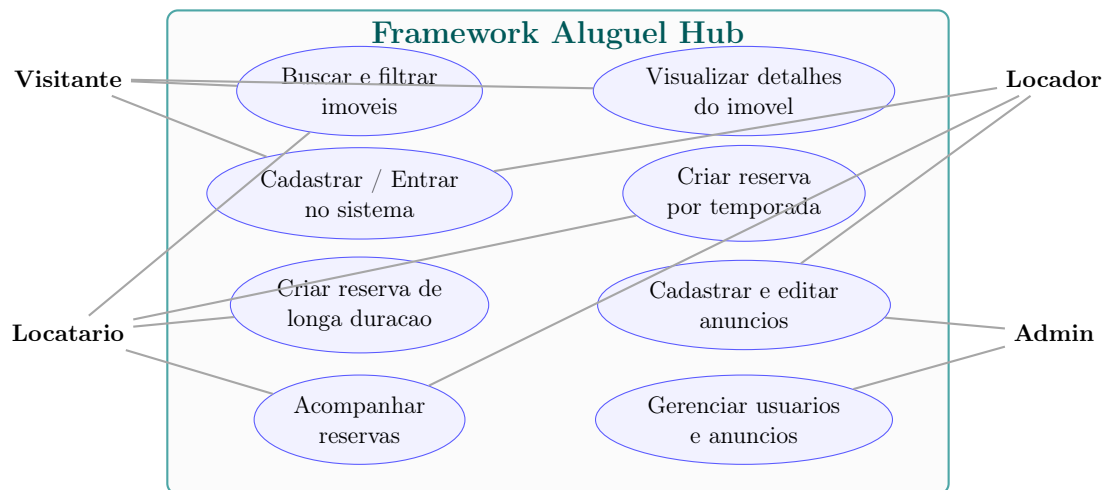


Figura 5: Diagrama de casos de uso do sistema de aluguel whitelabel.

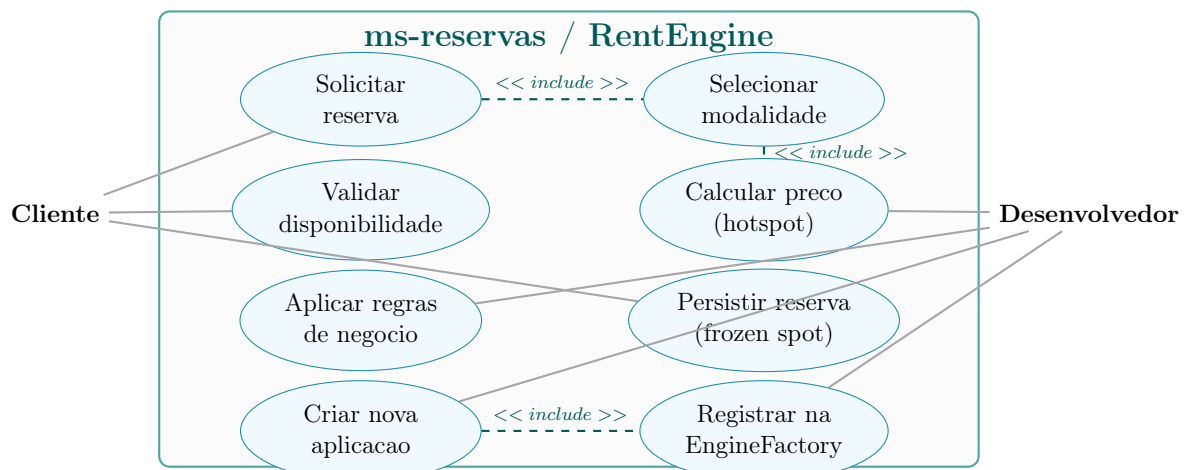


Figura 6: Diagrama de casos de uso do framework de reservas e seus hotspots.

4.2 Interfaces dos Componentes do Framework

A modelagem dos componentes considera suas interfaces de uso, isto é, os contratos publicos pelos quais cada componente é consumido dentro do framework.

4.3 Listagem dos Microservicos em Execucao

A tela principal do sistema, servida pelo Gateway em `http://localhost:8003`, destaca o uso dos microservicos por meio das rotas `/api/auth`, `/api/imoveis` e `/api/reservas`. Para evidenciar os servicos em execucao, cada microservico tambem expoe uma rota `/health`:

```
curl http://localhost:8000/health
# {"service":"ms-auth","status":"ok"}
```

Componente	Interface/Contrato	Uso no Framework
AuthController	register, login, me, validate, logout, refresh	Autenticacao, emissao e validacao de JWT para rotas protegidas.
CatalogController	index, show, store, update, destroy	Cadastro, consulta e manutencao dos imoveis disponiveis para reserva.
RentEngine	processReservation e hotspots protegidos	Classe do framework que fixa o fluxo de reserva e permite adaptacao por subclasses.
CreditCheckComponent	evaluate	Componente reutilizavel de analise de credito usado por composicao na modalidade de longa duracao.
EngineFactory	create, supportedModalities	Ponto de extensao que registra e instancia aplicacoes concretas do framework.

Tabela 4: Interfaces dos componentes usados no framework.

```

curl http://localhost:8001/health
# {"service":"ms-catalogo","status":"ok"}

curl http://localhost:8002/health
# {"service":"ms-reservas","status":"ok"}

curl http://localhost:8003
# Interface principal servida pelo API Gateway

```

Listing 2: Verificacao dos microsservicos em execucao e suas portas.

5 API Gateway

O *API Gateway* e o unico ponto de entrada do sistema (*Single Entry Point*). Suas caracteristicas principais sao:

- **Stateless:** nao mantem estado de sessao; cada requisicao carrega o token JWT necessario, repassado aos microsservicos.
- **Roteamento:** mapeia rotas externas (ex.: `/api/imoveis`) para os servicos internos correspondentes.
- **Comunicacao via cURL:** as chamadas internas sao feitas por HTTP com a extensao cURL do PHP nativo, mantendo baixo acoplamento.
- **Apresentacao:** serve as *views* (Blade/HTML) do front-end integrado.

5.1 Especificacao Documentada do API Gateway

O Gateway esta implementado em `gateway/src/Router.php`. Ele recebe toda requisicao externa, identifica o prefixo `/api/{servico}` e encaminha a chamada para o microsservico

correto usando `HttpClient`. O corpo da requisicao, o metodo HTTP e o cabeçalho `Authorization` sao preservados, mantendo a arquitetura stateless.

```
// Rotas iniciadas em /api/{servico}/... sao encaminhadas ao microservico.
// Rotas de escrita em imoveis/reservas exigem validacao JWT em ms-auth.
$isProtected = ($service === 'imoveis' && in_array($method, ['POST', 'PUT', 'DELETE'])
    ↪ )
    || ($service === 'reservas' && in_array($method, ['POST', 'PUT', 'DELETE'])
    ↪ );

if ($isProtected) {
    $authCheck = $this->http->forward('POST', $serviceMap['auth'] . '/validate', '',
    ↪ $headers);
    if ($authCheck['status'] !== 200) {
        $this->json(401, ['error' => 'Nao autorizado pelo Gateway.']);
        return;
    }
}
```

Listing 3: Trecho documentado do roteamento central do Gateway.

5.2 Microservico de Autenticacao (ms-auth)

O `ms-auth` (porta 8000, banco `db_auth`) e responsavel pela identidade do ecossistema. Implementa JWT no algoritmo HS256 de forma autocontida, expondo os endpoints `/register`, `/login`, `/me` e `/validate`. As senhas sao armazenadas com `bcrypt` e o acesso a dados usa PDO com *prepared statements*. Por ser stateless, o token viaja em cada requisicao (cabeçalho `Authorization`), sendo repassado pelo Gateway aos demais servicos.

Especificacao da API do ms-auth:

- POST `/api/auth/register`: cadastra usuario comum com papel locador ou locatario.
- POST `/api/auth/login`: autentica email/senha e retorna JWT.
- GET `/api/auth/me`: retorna dados do usuario autenticado.
- POST `/api/auth/validate`: valida token para o Gateway.
- POST `/api/auth/logout`: revoga token.
- POST `/api/auth/refresh`: emite novo token.

5.3 Microservico de Catalogo (ms-catalogo)

O `ms-catalogo` (porta 8001, banco `db_catalogo`) gerencia o cadastro de imoveis (casas e apartamentos) com uma API REST de CRUD: GET `/` (com filtros por cidade, tipo e disponibilidade), GET `/id`, POST `/`, PUT `/id` e DELETE `/id`. O relacionamento com o usuario proprietario (`owner_id`) e logico, sem chave estrangeira fisica, respeitando o isolamento do padrao *Database per Service*.

Especificacao da API do ms-catalogo:

- GET `/api/imoveis`: lista imoveis com filtros por cidade, tipo, disponibilidade, paginação e ordenação.
- GET `/api/imoveis/{id}`: detalha um imóvel.
- POST `/api/imoveis`: cria anúncio. Rota protegida para admin ou locador.
- PUT `/api/imoveis/{id}`: atualiza anúncio. Exige dono ou administrador.
- DELETE `/api/imoveis/{id}`: remove anúncio. Exige dono ou administrador.

5.4 Microserviço de Reservas (ms-reservas)

O `ms-reservas` (porta 8002, banco `db_reservas`) contém a classe `RentEngine`, que é a classe de framework. A API recebe uma reserva e seleciona a aplicação concreta pelo campo `modality`. As modalidades disponíveis são `vacation` e `long_term`.

Especificação da API do `ms-reservas`:

- GET `/api/reservas`: lista reservas e informa modalidades suportadas.
- GET `/api/reservas/{id}`: detalha uma reserva.
- POST `/api/reservas`: cria reserva usando a engine escolhida por `modality`. Rota protegida por token JWT no Gateway.

6 Front-End Integrado

A camada de apresentação é servida pelo próprio Gateway (porta 8003). O design foi derivado via **Figma MCP** (`get_figma_data`), mesclando dois kits da Figma Community com paleta de cores própria (identidade UFAL):

- **M-Rent** (Property Management): cards de imóvel com *border-radius* 12px, badges de tipo e estrutura de listagem com filtros.
- **Estatery** (Real Estate SaaS UI Kit): *hero*, *Search Bar*, abas por tipo de imóvel e barra de resultados.

A view consome exclusivamente as rotas `/api/*`:

- **Autenticacao**: login via POST `/api/auth/login`; token JWT no `localStorage`, reenviado em `Authorization`.
- **Catalogo**: busca com filtros de API (`city`, `type`, `available`) e filtros client-side (`preco`, `quartos`, `area`, `ordenacao`).
- **Reservas**: criação via POST `/api/reservas` com modalidades `vacation` e `long_term`.

Assets em `gateway/public/assets/css/app.css` e `gateway/views/home.php`.

7 Tecnologias Utilizadas

7.1 Tecnologias dos Microservicos

Servico	Linguagem	Banco	Tecnologias
gateway	PHP 8.2	–	Servidor embutido do PHP, cURL, HTML, CSS e JavaScript.
ms-auth	PHP 8.2	MySQL db_auth	PDO, JWT HS256, bcrypt, Composer e variaveis .env.
ms-catalogo	PHP 8.2	MySQL db_catalogo	PDO, API REST, filtros, paginacao, autorizacao por cabecalhos validados.
ms-reservas	PHP 8.2	MySQL db_reservas	Template Method, classes abstratas, heranca, composicao e Factory.

Tabela 5: Tecnologias por microservico.

7.2 Tecnologias do Framework e das Aplicacoes

O framework foi implementado com PHP orientado a objetos, usando namespaces, `strict_types`, classes `final`, classe abstrata, DTOs e repositorios. As aplicacoes construidas por reuso sao as modalidades `VacationRent` e `LongTermRent`. Ambas reutilizam a mesma engine, a mesma API de reservas, a mesma persistencia e a mesma integracao pelo Gateway.

No front-end, a interface usa HTML, CSS e JavaScript servidos pelo Gateway. A interface principal permite autenticar o usuario, listar imoveis, aplicar filtros e criar reservas, evidenciando o uso integrado dos microservicos.

8 Template Method e Hotspots

O nucleo de reuso e a classe abstrata `RentEngine`. Ela implementa o **Template Method** `processReservation()`, que fixa a sequencia invariante de passos do processo de reserva. Os **hotspots** (metodos abstratos ou *hooks*) representam os pontos de variacao implementados por cada subclasse.

Frozen Spots Passos fixos: validacao de disponibilidade, persistencia da reserva e confirmacao. Sao implementados na classe abstrata e nao mudam.

Hot Spots Passos variaveis: calculo de preco, regras de validacao especificas e etapas opcionais (ex.: analise de credito). Sao abstratos ou *hooks*, definidos pelas subclases.

```
abstract class RentEngine
{
    // TEMPLATE METHOD: esqueleto invariante, final (nao sobrescrevivel)
```

```

final public function processReservation(ReservationRequest $req):
↳ ReservationResult
{
    $this->validateBaseRequest($req); // (1) FROZEN
    $this->ensureAvailability($req); // (2) FROZEN
    $price = $this->calculatePrice($req); // (3) HOTSPOT (abstrato)
    $this->applyBusinessRules($req, $price); // (4) HOTSPOT (hook)
    $status = $this->resolveStatus($req, $price); // (5) HOTSPOT (hook)
    $id = $this->repository->create( // (6) FROZEN: persistencia
        $req, $this->modalityName(), $price, $status, $this->reservationNotes($req)
    );
    $this->afterReservation($id, $req, $price); // (7) HOTSPOT (hook)
    return new ReservationResult(/* ... */);
}

// HOTSPOTS obrigatorios (abstratos)
abstract protected function modalityName(): string;
abstract protected function calculatePrice(ReservationRequest $req): float;

// HOTSPOTS opcionais (hooks com comportamento default)
protected function applyBusinessRules(ReservationRequest $req, float $price): void
↳ {}
protected function resolveStatus(ReservationRequest $req, float $price): string {
↳ return 'confirmed'; }
protected function afterReservation(int $id, ReservationRequest $req, float $price)
↳ : void {}
}

```

Listing 4: Template Method implementado em ms-reservas/src/Engine/RentEngine.php.

A selecao da modalidade concreta ocorre em **EngineFactory**, ponto unico de extensao do framework: registrar uma nova modalidade nao exige alterar o nucleo, respeitando o *Open/Closed Principle*.

8.1 Classes Concretas e Prova de Reuso

Duas modalidades concretas instanciam o framework, reaproveitando o mesmo Template Method e variando apenas os hotspots:

VacationRent (temporada) Reuso por **heranca**: sobrescreve somente os hotspots obrigatorios (**modalityName** e **calculatePrice**), aplicando adicional de alta temporada para estadias curtas e taxa de limpeza. Os hooks opcionais usam o comportamento default da **RentEngine**.

LongTermRent (longa duracao) Combina **heranca** e **composicao**: calcula o preco mensal com desconto para contratos longos e injeta o **CreditCheckComponent** no hook **applyBusinessRules**. O resultado da analise de credito governa o hook **resolveStatus** (**confirmed** se aprovado, **pending** caso contrario).

A Tabela 6 resume como cada modalidade exercita os pontos do framework.

```

final class LongTermRent extends RentEngine
{
    public function __construct(

```

Ponto do framework	VacationRent	LongTermRent
calculatePrice (abstract)	sobrescrito (diaria)	sobrescrito (mensal)
applyBusinessRules (hook)	default (vazio)	credit check (composicao)
resolveStatus (hook)	default (confirmed)	dinamico (credito)
Esqueleto (frozen spots)	reutilizado	reutilizado

Tabela 6: Reuso dos pontos do framework por modalidade.

```

    ReservationRepository $repository,
    private readonly CreditCheckComponent $creditCheck = new CreditCheckComponent()
) { parent::__construct($repository); }

protected function applyBusinessRules(ReservationRequest $req, float $price): void
↳ {
    $this->creditResult = $this->creditCheck->evaluate($req); // COMPOSICAO
}
protected function resolveStatus(ReservationRequest $req, float $price): string {
    return $this->creditResult['approved'] ? 'confirmed' : 'pending';
}
}

```

Listing 5: Reuso por composicao: credit check no hook do LongTermRent.

Na Etapa 6, *VacationRent* sobrescreve *calculatePrice()* com tarifas de temporada, enquanto *LongTermRent* adiciona, por composicao, o *CreditCheckComponent* dentro do *hook* *applyBusinessRules()*, demonstrando a combinacao de reuso por heranca e por composicao.

8.2 Aplicacao 1: Aluguel por Temporada

A primeira aplicacao concreta e *VacationRent*. Ela representa plataformas de aluguel por temporada, como imoveis para poucos dias. A aplicacao reutiliza os frozen spots da *RentEngine* e implementa o hotspot *calculatePrice()* com diarias, taxa de limpeza e adicional para estadias curtas.

```

final class VacationRent extends RentEngine
{
    protected function modalityName(): string
    {
        return 'vacation';
    }

    protected function calculatePrice(ReservationRequest $request): float
    {
        $nights = $request->nights();
        $subtotal = $nights * $request->dailyRate;

        if ($nights <= 3) {
            $subtotal *= 1.15;
        }

        return $subtotal + 120.0;
    }
}

```



```
}  
}
```

Listing 6: Hotspots implementados na aplicacao VacationRent.

8.3 Aplicacao 2: Aluguel de Longa Duracao

A segunda aplicacao concreta e `LongTermRent`. Ela representa contratos residenciais longos. Alem de implementar o preco mensal, usa composicao com `CreditCheckComponent` para adaptar o processo pelo hotspot `applyBusinessRules()`.

```
final class LongTermRent extends RentEngine  
{  
    protected function modalityName(): string  
    {  
        return 'long_term';  
    }  
  
    protected function calculatePrice(ReservationRequest $request): float  
    {  
        $months = $request->months();  
        $subtotal = $months * $request->monthlyRate;  
        return $months >= 12 ? $subtotal * 0.90 : $subtotal;  
    }  
  
    protected function applyBusinessRules(ReservationRequest $request, float $price):  
        ↪ void  
    {  
        $this->creditResult = $this->creditCheck->evaluate($request);  
    }  
  
    protected function resolveStatus(ReservationRequest $request, float $price): string  
    {  
        return ($this->creditResult['approved'] ?? false) ? 'confirmed' : 'pending';  
    }  
}
```

Listing 7: Hotspots implementados na aplicacao LongTermRent.

8.4 Como Criar uma Nova Aplicacao Reusando o Framework

Para criar uma nova aplicacao, por exemplo aluguel corporativo, o desenvolvedor deve:

1. Criar uma classe concreta em `ms-reservas/src/Modalities`.
2. Herdar de `RentEngine`.
3. Implementar os hotspots obrigatorios `modalityName()` e `calculatePrice()`.
4. Sobrescrever hooks opcionais se a regra exigir validacao adicional, status diferente, observacoes ou pos-processamento.
5. Registrar a nova modalidade em `EngineFactory`.

```
final class CorporateRent extends RentEngine
{
    protected function modalityName(): string
    {
        return 'corporate';
    }

    protected function calculatePrice(ReservationRequest $request): float
    {
        $months = $request->months();
        $base = $months * $request->monthlyRate;
        return $base + 500.0; // taxa de administracao corporativa
    }

    protected function applyBusinessRules(ReservationRequest $request, float $price):
    ↪ void
    {
        // Hotspot opcional: validar CNPJ, contrato empresarial ou limite de ocupacao.
    }
}

// Registro na fabrica:
return match ($modality) {
    'vacation' => new VacationRent($repository),
    'long_term' => new LongTermRent($repository),
    'corporate' => new CorporateRent($repository),
};
```

Listing 8: Exemplo de nova aplicacao por reuso do framework.

O restante do processo permanece reaproveitado: validacao base, disponibilidade, persistencia, retorno padronizado e exposicao via POST /api/reservas.

9 Guia do Desenvolvedor

9.1 Pre-requisitos

PHP 8.2+ (extensoes pdo_mysql, curl, json), MySQL 8.0+ e Composer 2.x.

9.2 Subindo os Servicos

Em terminais separados:

```
cd ms-auth && composer install && php -S localhost:8000 -t public
cd ms-catalogo && composer install && php -S localhost:8001 -t public
cd ms-reservas && composer install && php -S localhost:8002 -t public
cd gateway && composer install && php -S localhost:8003 -t public
```

9.3 Criando os Bancos

```
mysql -u root -p < ms-auth/sql/db_auth.sql
mysql -u root -p < ms-catalogo/sql/db_catalogo.sql
mysql -u root -p < ms-reservas/sql/db_reservas.sql
```

9.4 Como Estender o Framework

Para criar uma nova modalidade de aluguel, basta criar uma subclasse de `RentEngine` e implementar os *hotspots*, sem alterar o fluxo principal, respeitando o *Open/Closed Principle*.

10 Funcionamento do Sistema

10.1 Passo a Passo das Telas

O funcionamento do sistema ocorre a partir da interface principal em <http://localhost:8003>. A tela inicial é servida pelo Gateway e consome os microserviços por rotas `/api/*`.

1. **Tela inicial e busca:** o usuário visualiza a listagem de imóveis e aplica filtros por cidade, tipo, preço, quartos, área e disponibilidade. A tela usa o `ms-catalogo` via `GET /api/imoveis`.
2. **Login:** o usuário informa e-mail e senha. O Gateway envia `POST /api/auth/login` ao `ms-auth`; o token retornado fica no `localStorage`.
3. **Cadastro/edição de imóvel:** usuários `locador` ou `admin` enviam dados do anúncio. O Gateway valida o token no `ms-auth` e encaminha ao `ms-catalogo`.
4. **Reserva por temporada:** a tela envia `modality: vacation`. O `ms-reservas` aciona `VacationRent`, calculando diárias, taxa de limpeza e adicional de alta temporada.
5. **Reserva de longa duração:** a tela envia `modality: long_term`. O `ms-reservas` aciona `LongTermRent`, calcula mensalidades e chama o componente de análise de crédito.

10.2 Exemplos de Execução das Duas Aplicações

```
curl -X POST http://localhost:8003/api/reservas \
-H "Content-Type: application/json" \
-H "Authorization: Bearer TOKEN" \
-d '{
  "modality": "vacation",
  "property_id": 1,
  "check_in": "2026-07-10",
  "check_out": "2026-07-13",
  "daily_rate": 350,
  "guests": 2
}'
```

Listing 9: Criacao de reserva por temporada.

```
curl -X POST http://localhost:8003/api/reservas \  
-H "Content-Type: application/json" \  
-H "Authorization: Bearer TOKEN" \  
-d '{  
  "modality": "long_term",  
  "property_id": 1,  
  "check_in": "2026-08-01",  
  "check_out": "2027-08-01",  
  "monthly_rate": 4000,  
  "guests": 2,  
  "extras": { "monthly_income": 13000 }  
}'
```

Listing 10: Criacao de reserva de longa duracao.

Esses dois exemplos demonstram que as aplicacoes mudam apenas os hotspots, enquanto o fluxo principal permanece o mesmo.

11 Prints do Sistema em Execucao

As figuras a seguir foram capturadas com os quatro servicos em execucao (`ms-auth`, `ms-catalogo`, `ms-reservas` e `gateway`) e com os bancos provisionados pelos scripts SQL do projeto.

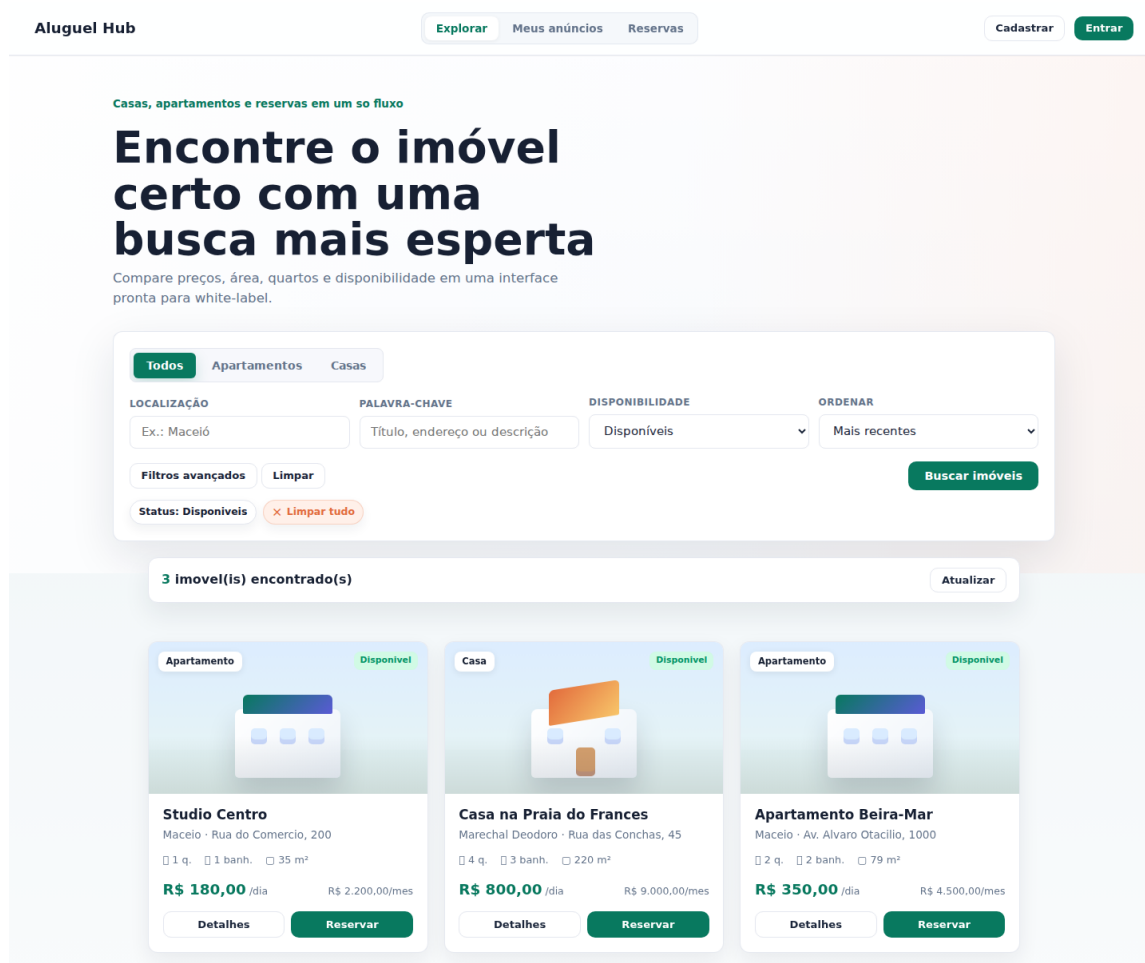


Figura 7: Tela inicial de exploração e busca de imóveis.

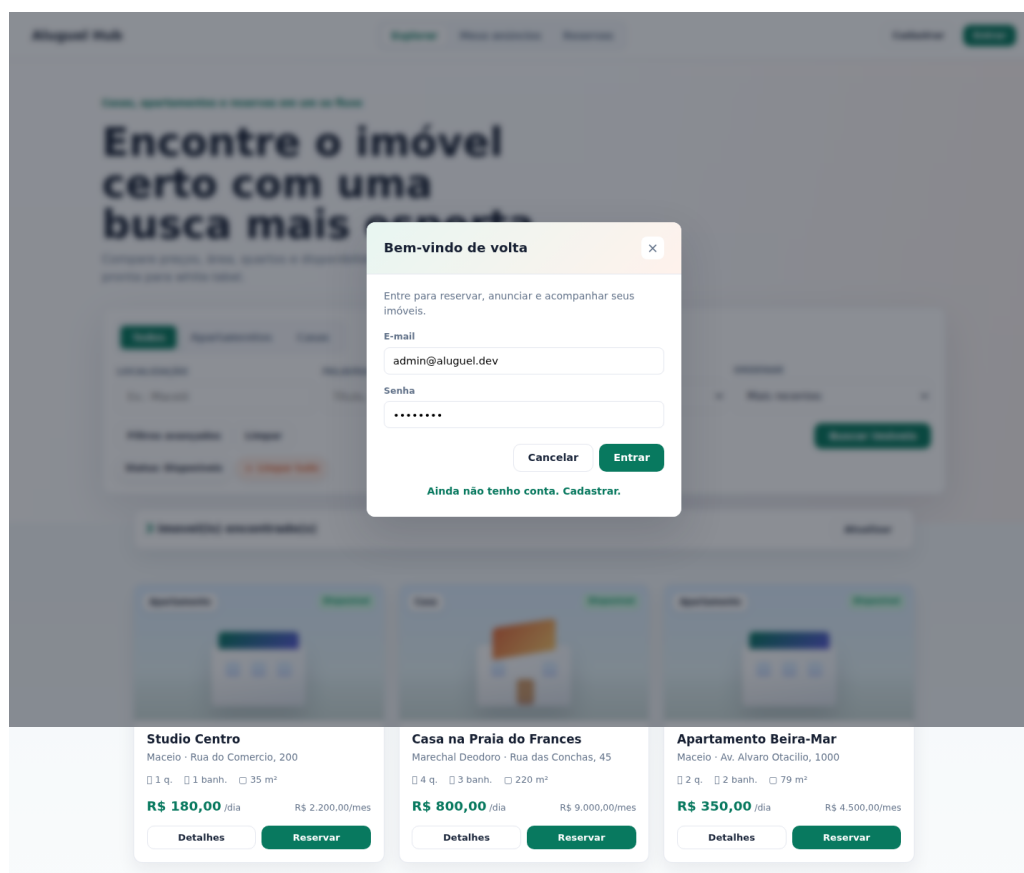


Figura 8: Modal de login consumindo o microservico de autenticao.

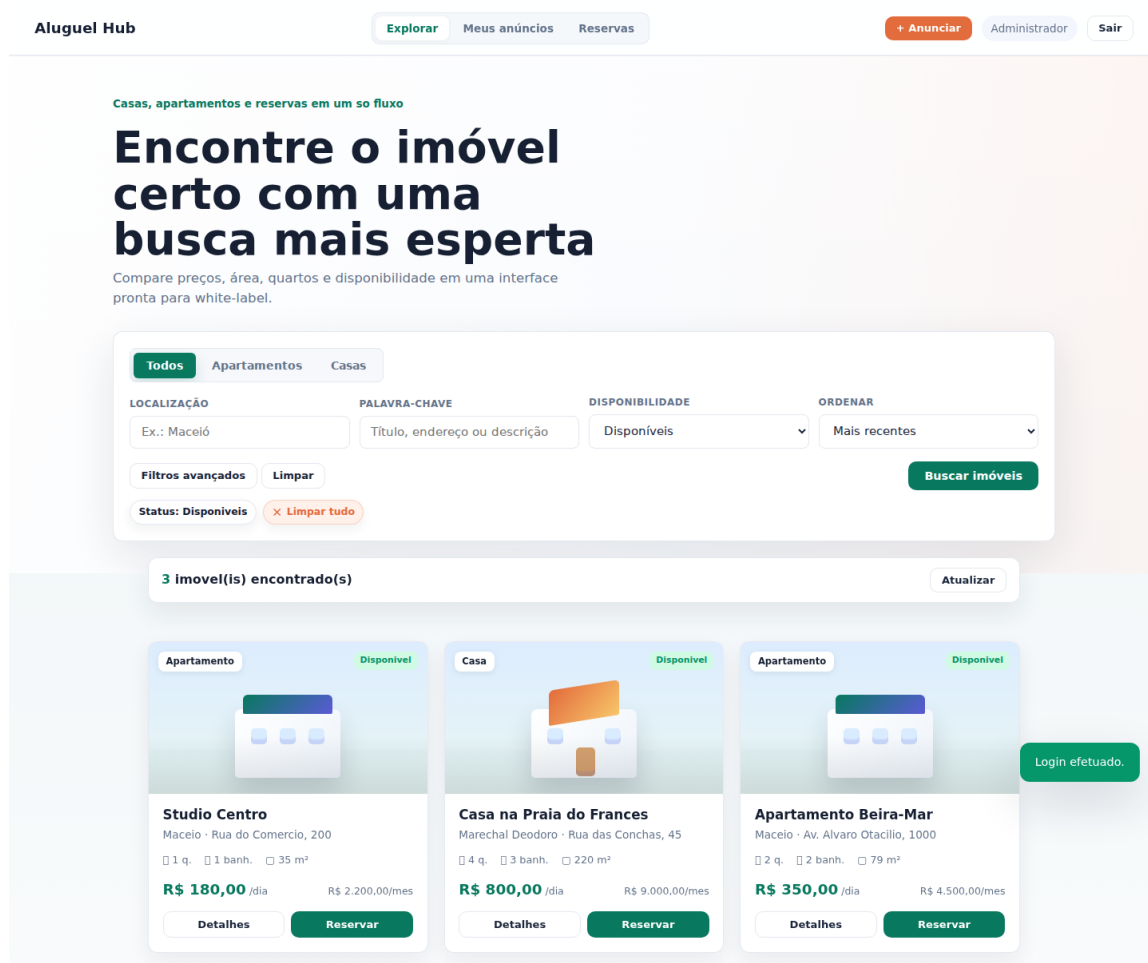


Figura 9: Tela principal após autenticação do usuário administrador.

O modal de cadastro, intitulado "Criar conta", permite que o usuário registre-se para reservar imóveis ou publicar anúncios. Ele contém campos para Nome, E-mail, Senha (com validação de 6 caracteres) e Perfil (com uma lista suspensa selecionando "Quero reservar imóveis"). Botões para "Cancelar" e "Criar conta" estão no rodapé, assim como um link para quem já possui uma conta. À direita, uma seção incentivadora "Comece com seu perfil certo" destaca a facilidade de reserva para locatários. O fundo do modal mostra uma pré-visualização de cartões de imóveis, incluindo o "Studio Centro", a "Casa na Praia do Frances" e o "Apartamento Beira-Mar".

Figura 10: Modal de cadastro de usuário com escolha de perfil.

A tela de detalhes do imóvel, acessível via botão "Voltar", apresenta o anúncio do "Apartamento Beira-Mar" em Maceio. O imóvel é rotulado como "Disponível". A interface destaca as características principais: 2 Quartos, 2 Banheiros, 79 m² e o código #1. A descrição menciona uma vista para o mar e mobília completa. Um resumo da locação mostra o preço de R\$ 350,00 por dia (R\$ 4.500,00 por mês). Botões de ação incluem "Reservar agora" e "Editar anuncio". O cabeçalho da página contém links para "Explorar", "Meus anúncios", "Reservas", "Anunciar", "Administrador" e "Sair".

Figura 11: Tela de detalhes do imóvel e chamada para reserva.

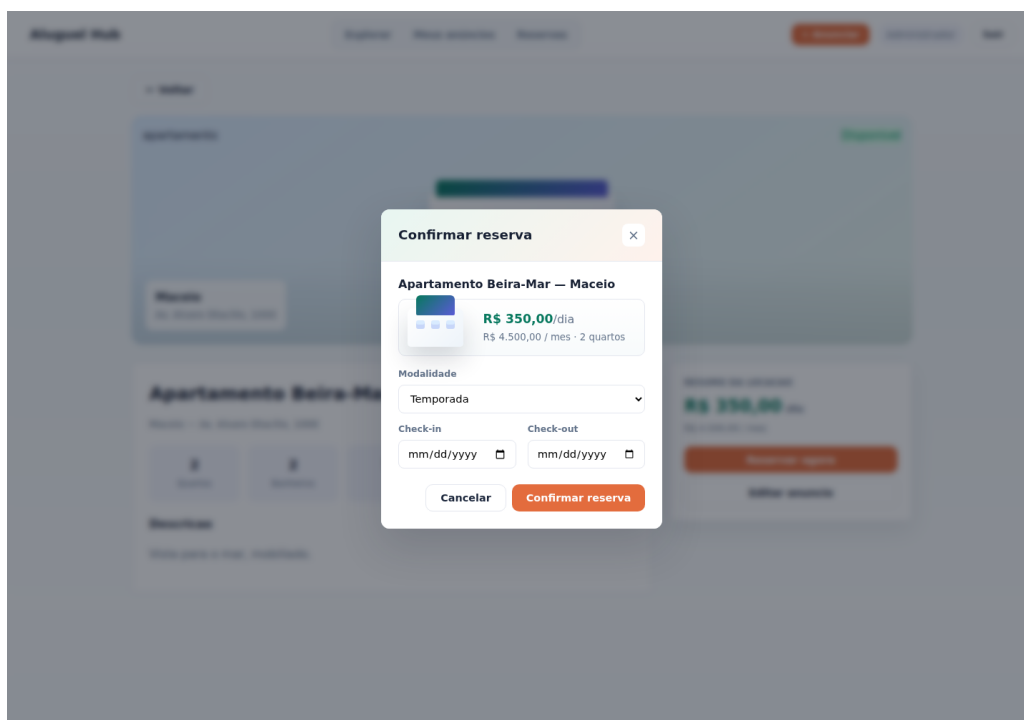


Figura 12: Modal de criação de reserva usando as modalidades do framework.

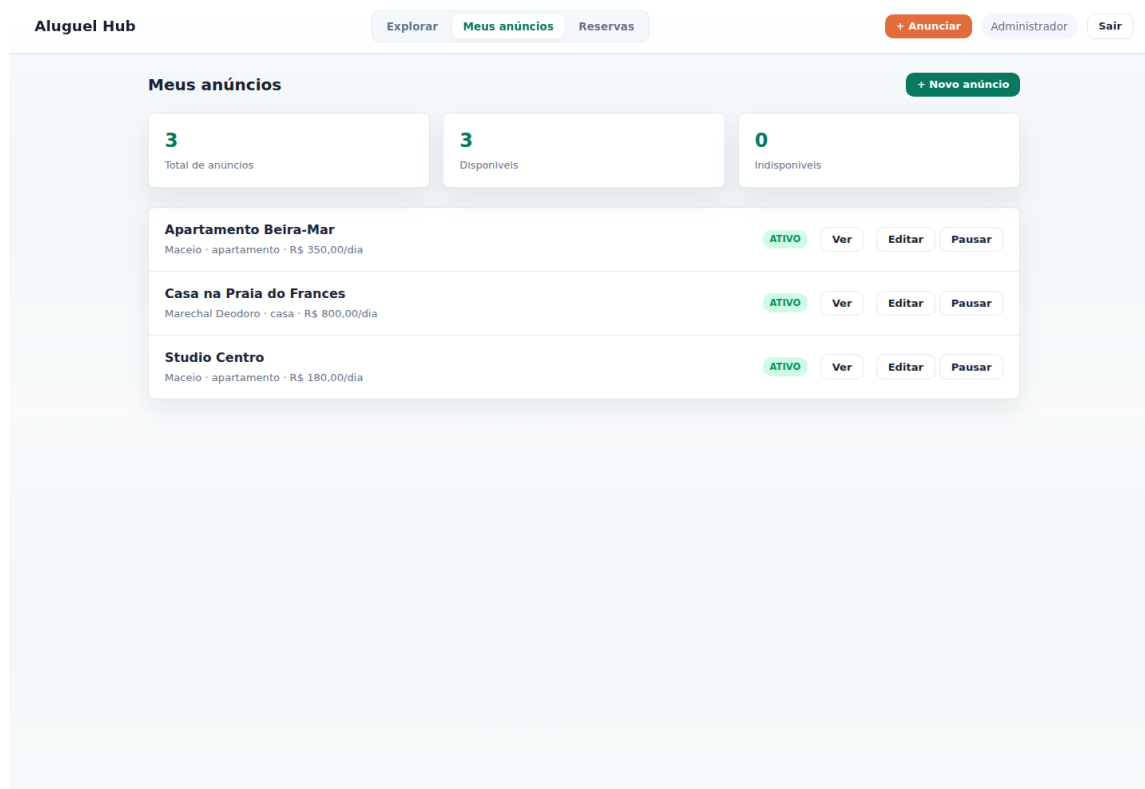
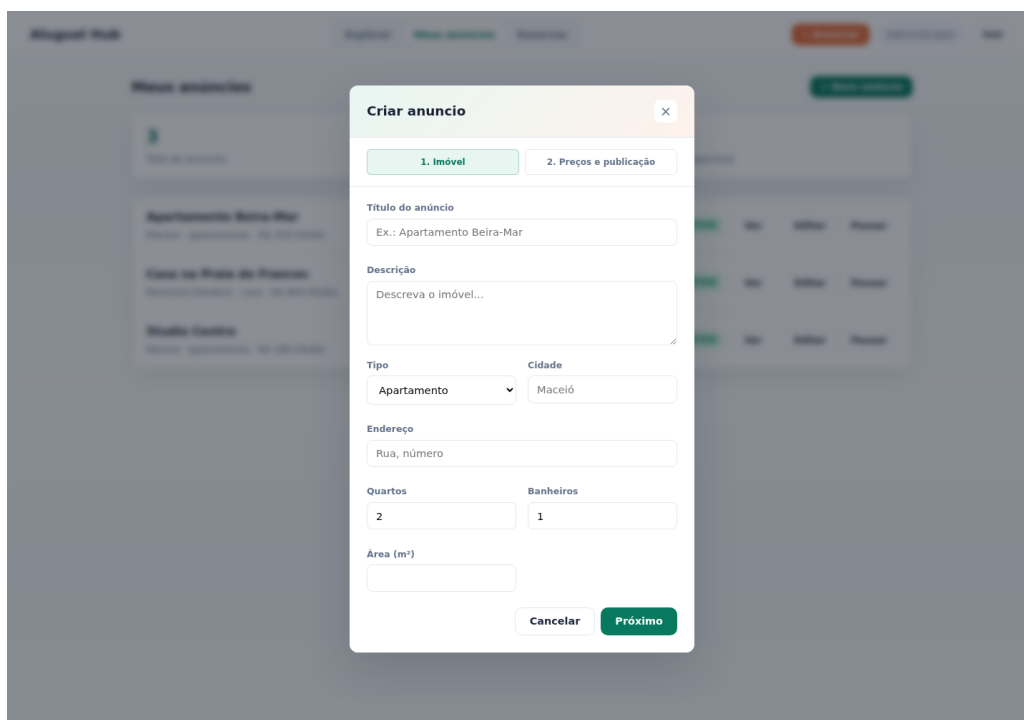
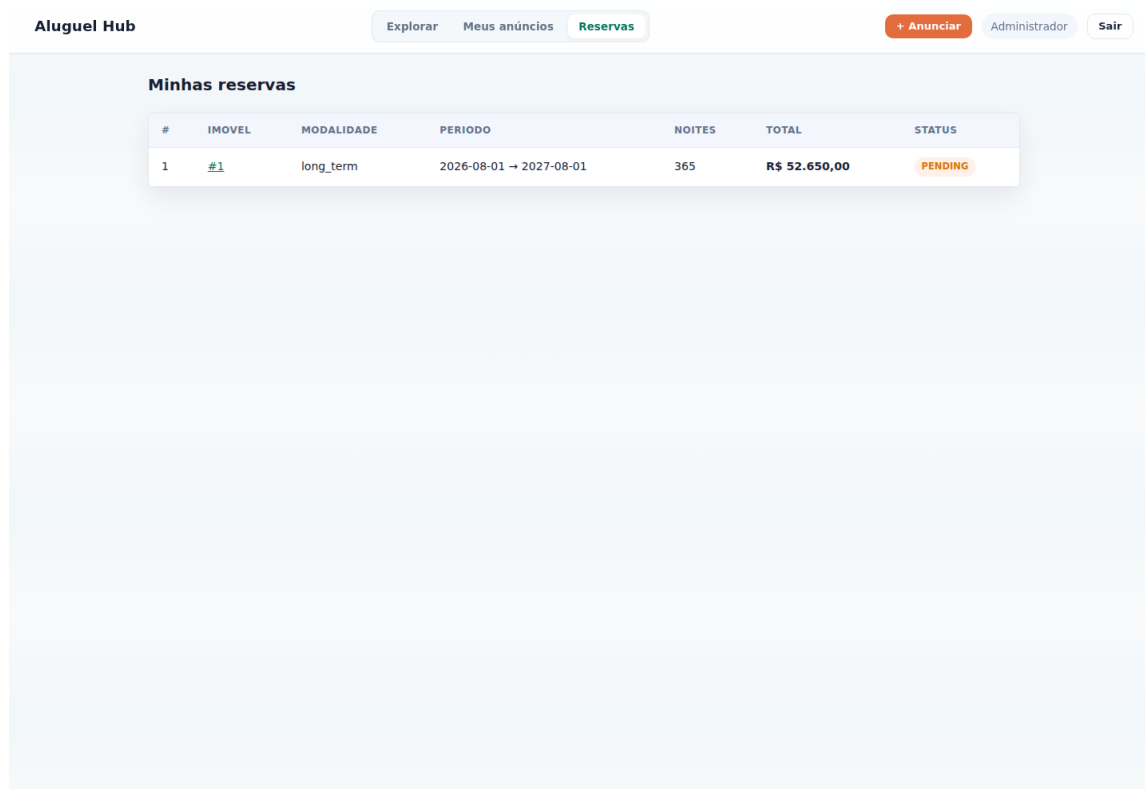


Figura 13: Tela de anúncios do usuário autenticado.



Modal de cadastro de novo anúncio. O formulário é dividido em duas etapas: 1. Imóvel e 2. Preços e publicação. A etapa 1 contém campos para: Título do anúncio (Ex.: Apartamento Beira-Mar), Descrição (Descreva o imóvel...), Tipo (dropdown com opção Apartamento), Cidade (Maceió), Endereço (Rua, número), Quartos (2) e Banheiros (1). Há também um campo para Área (m²). Botões de Cancelar e Próximo estão na base.

Figura 14: Modal de cadastro de novo anúncio.



Tela de reservas com uma reserva de longa duração criada pelo framework. O cabeçalho mostra "Aluguel Hub" e opções de navegação: Explorar, Meus anúncios, Reservas, + Anunciar, Administrador e Sair.

#	IMÓVEL	MODALIDADE	PERÍODO	NOITES	TOTAL	STATUS
1	#1	long_term	2026-08-01 → 2027-08-01	365	R\$ 52.650,00	PENDING

Figura 15: Tela de reservas com uma reserva de longa duração criada pelo framework.

12 Repositorio GitHub

O código completo do projeto, incluindo framework, aplicações concretas, microsserviços, scripts SQL, front-end e relatório, está disponível no GitHub:

<https://github.com/TheMarksan/aluguel-services-framework>

As instruções de execução estão no `README.md`: instalar dependências com Composer, criar os bancos com os scripts SQL, configurar os arquivos `.env` a partir dos `.env.example` e subir cada microsserviço em sua porta.

12.1 Observação sobre Vídeo

O requisito de vídeo de execução é aplicável apenas a frameworks para jogos. Como este projeto é um framework whitelabel para sistemas de aluguel de imóveis usando microsserviços, esse item não se aplica.

A Anexos: Código dos Microsserviços

Esta seção referencia o código documentado dos principais arquivos de cada microsserviço implementado. Os arquivos completos também estão no repositório GitHub.

A.1 API Gateway

gateway/public/index.php

Arquivo de código disponível no repositório GitHub:

<https://github.com/TheMarksan/aluguel-services-framework>

Caminho no projeto: `../gateway/public/index.php`

gateway/src/Router.php

Arquivo de código disponível no repositório GitHub:

<https://github.com/TheMarksan/aluguel-services-framework>

Caminho no projeto: `../gateway/src/Router.php`

gateway/src/HttpClient.php

Arquivo de código disponível no repositório GitHub:

<https://github.com/TheMarksan/aluguel-services-framework>

Caminho no projeto: `../gateway/src/HttpClient.php`

gateway/src/Config.php

Arquivo de código disponível no repositório GitHub:

<https://github.com/TheMarksan/aluguel-services-framework>

Caminho no projeto: `../gateway/src/Config.php`

A.2 Microservico ms-auth

ms-auth/public/index.php

Arquivo de código disponível no repositório GitHub:
<https://github.com/TheMarksan/aluguel-services-framework>
Caminho no projeto: ../ms-auth/public/index.php

ms-auth/src/AuthController.php

Arquivo de código disponível no repositório GitHub:
<https://github.com/TheMarksan/aluguel-services-framework>
Caminho no projeto: ../ms-auth/src/AuthController.php

ms-auth/src/UserRepository.php

Arquivo de código disponível no repositório GitHub:
<https://github.com/TheMarksan/aluguel-services-framework>
Caminho no projeto: ../ms-auth/src/UserRepository.php

ms-auth/src/Jwt.php

Arquivo de código disponível no repositório GitHub:
<https://github.com/TheMarksan/aluguel-services-framework>
Caminho no projeto: ../ms-auth/src/Jwt.php

ms-auth/src/Database.php

Arquivo de código disponível no repositório GitHub:
<https://github.com/TheMarksan/aluguel-services-framework>
Caminho no projeto: ../ms-auth/src/Database.php

ms-auth/sql/db_auth.sql

Arquivo de código disponível no repositório GitHub:
<https://github.com/TheMarksan/aluguel-services-framework>
Caminho no projeto: ../ms-auth/sql/db_auth.sql

A.3 Microservico ms-catalogo

ms-catalogo/public/index.php

Arquivo de código disponível no repositório GitHub:
<https://github.com/TheMarksan/aluguel-services-framework>
Caminho no projeto: ../ms-catalogo/public/index.php

ms-catalogo/src/CatalogController.php

Arquivo de código disponível no repositório GitHub:
<https://github.com/TheMarksan/aluguel-services-framework>
Caminho no projeto: ../ms-catalogo/src/CatalogController.php

ms-catalogo/src/PropertyRepository.php

Arquivo de código disponível no repositório GitHub:
<https://github.com/TheMarksan/aluguel-services-framework>
Caminho no projeto: ../ms-catalogo/src/PropertyRepository.php

ms-catalogo/src/Database.php

Arquivo de código disponível no repositório GitHub:
<https://github.com/TheMarksan/aluguel-services-framework>
Caminho no projeto: ../ms-catalogo/src/Database.php

ms-catalogo/sql/db_catalogo.sql

Arquivo de código disponível no repositório GitHub:
<https://github.com/TheMarksan/aluguel-services-framework>
Caminho no projeto: ../ms-catalogo/sql/db_catalogo.sql

A.4 Microserviço ms-reservas e Aplicações**ms-reservas/public/index.php**

Arquivo de código disponível no repositório GitHub:
<https://github.com/TheMarksan/aluguel-services-framework>
Caminho no projeto: ../ms-reservas/public/index.php

ms-reservas/src/ReservationController.php

Arquivo de código disponível no repositório GitHub:
<https://github.com/TheMarksan/aluguel-services-framework>
Caminho no projeto: ../ms-reservas/src/ReservationController.php

ms-reservas/src/Engine/RentEngine.php

Arquivo de código disponível no repositório GitHub:
<https://github.com/TheMarksan/aluguel-services-framework>
Caminho no projeto: ../ms-reservas/src/Engine/RentEngine.php

ms-reservas/src/Engine/EngineFactory.php

Arquivo de código disponível no repositório GitHub:
<https://github.com/TheMarksan/aluguel-services-framework>
Caminho no projeto: ../ms-reservas/src/Engine/EngineFactory.php

ms-reservas/src/Modalities/VacationRent.php

Arquivo de código disponível no repositório GitHub:
<https://github.com/TheMarksan/aluguel-services-framework>
Caminho no projeto: ../ms-reservas/src/Modalities/VacationRent.php

ms-reservas/src/Modalities/LongTermRent.php

Arquivo de código disponível no repositório GitHub:
<https://github.com/TheMarksan/aluguel-services-framework>
Caminho no projeto: ../ms-reservas/src/Modalities/LongTermRent.php

ms-reservas/src/Components/CreditCheckComponent.php

Arquivo de código disponível no repositório GitHub:
<https://github.com/TheMarksan/aluguel-services-framework>
Caminho no projeto: ../ms-reservas/src/Components/CreditCheckComponent.php

ms-reservas/src/ReservationRepository.php

Arquivo de código disponível no repositório GitHub:

<https://github.com/TheMarksan/aluguel-services-framework>

Caminho no projeto: `../ms-reservas/src/ReservationRepository.php`

`ms-reservas/sql/db_reservas.sql`

Arquivo de código disponível no repositório GitHub:

<https://github.com/TheMarksan/aluguel-services-framework>

Caminho no projeto: `../ms-reservas/sql/db_reservas.sql`

Referencias

Referências

- [1] GAMMA, E. et al. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
- [2] SOMMERVILLE, I. *Engenharia de Software*. 10. ed. Pearson, 2018.
- [3] NEWMAN, S. *Building Microservices*. 2. ed. O'Reilly, 2021.
- [4] SCHWABER, K.; SUTHERLAND, J. *The Scrum Guide*. 2020.
- [5] FOWLER, M. *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.
- [6] PHP DOCUMENTATION. *PHP 8.2 Manual*. Disponível em: <https://www.php.net/manual/>. Acesso em: 2026.